

“Año de la Esperanza y el Fortalecimiento de la Democracia”

**UNIVERSIDAD PERUANA LOS ANDES**  
**FACULTAD DE INGENIERÍA**

**ESCUELA PROFESIONAL DE SISTEMAS Y COMPUTACIÓN**



Tema: RESUMEN DE FUNDAMENTOS DE LA  
ARQUITECTURA DE SOFTWARE

Estudiante: QUISPE OCHOA JEFFRY SERGIO

Huancayo – Perú

2026

## INTRODUCCIÓN

En el desarrollo de software actual, es fundamental comprender cómo se estructuran los sistemas antes de implementarlos. En este contexto, la arquitectura de software permite organizar y definir los componentes de un sistema, así como las relaciones entre ellos, asegurando que el producto final sea eficiente, seguro y adaptable.

Como estudiante de Ingeniería de Sistemas, entender estos fundamentos me permite analizar cómo se diseñan soluciones tecnológicas desde un nivel estratégico, considerando tanto los requisitos funcionales como los no funcionales. Esto resulta clave para desarrollar sistemas de calidad que puedan evolucionar en el tiempo.

# FUNDAMENTOS DE LA ARQUITECTURA DE SOFTWARE

## 1. CONCEPTO Y OBJETIVOS DE LA ARQUITECTURA DE SOFTWARE

La arquitectura de software representa la estructura base de un sistema, incluyendo sus componentes, interacciones y principios de diseño. De acuerdo con estándares internacionales, su propósito es ofrecer una visión clara que facilite la toma de decisiones durante el desarrollo.

### Objetivos principales:

- Definir una estructura organizada del sistema.
- Garantizar el cumplimiento de requisitos funcionales y no funcionales.
- Reducir riesgos técnicos y de negocio.
- Facilitar decisiones sobre tecnologías y herramientas.

### Ejemplo

En una aplicación bancaria moderna, la arquitectura puede estar basada en microservicios, donde cada módulo (usuarios, pagos, seguridad) funciona de manera independiente, permitiendo escalabilidad y alta disponibilidad.

## 2. IMPORTANCIA EN EL CICLO DE VIDA DEL SOFTWARE

La arquitectura es clave en todas las etapas del desarrollo:

- **Análisis:** permite interpretar los requisitos.
- **Diseño:** define modelos estructurales.
- **Construcción:** guía la implementación.
- **Pruebas:** ayuda a identificar qué validar.
- **Mantenimiento:** facilita futuras mejoras.

Una mala arquitectura genera sistemas difíciles de mantener, mientras que una buena arquitectura permite evolución continua.

### Ejemplo

En un sistema de ventas online, separar módulos como catálogo, pagos y usuarios permite modificar cada parte sin afectar al resto.

### 3. ESTÁNDARES INTERNACIONALES

Los estándares ayudan a mantener calidad y coherencia en el desarrollo.

- **ISO/IEC/IEEE 42010:** guía la documentación arquitectónica.
- **TOGAF:** orienta el diseño de arquitecturas empresariales.
- **SEI (ATAM):** evalúa decisiones arquitectónicas según atributos de calidad.

Estos estándares permiten justificar decisiones y mejorar la comunicación entre equipos.

### 4. ROL DEL ARQUITECTO DE SOFTWARE

El arquitecto es responsable de diseñar y supervisar la estructura del sistema.

#### Funciones principales:

- Definir la arquitectura general.
- Seleccionar patrones adecuados.
- Asegurar calidad del sistema.
- Documentar decisiones técnicas.
- Guiar al equipo de desarrollo.

#### Ejemplo

En una startup, el arquitecto puede decidir usar microservicios para permitir crecimiento rápido y escalabilidad.

## PRINCIPIOS DE DISEÑO ARQUITECTÓNICO

### 5. MODULARIDAD, COHESIÓN Y ACOPLAMIENTO

- **Modularidad:** dividir el sistema en partes independientes.
- **Cohesión:** cada módulo cumple una función específica.
- **Acoplamiento:** los módulos deben depender lo menos posible entre sí.

#### Ejemplo

En un sistema hospitalario, separar módulos como pacientes y facturación mejora la organización y reduce errores.

## 6. ESCALABILIDAD, MANTENIBILIDAD Y REUTILIZACIÓN

- **Escalabilidad:** capacidad de crecer según la demanda.
- **Mantenibilidad:** facilidad para modificar el sistema.
- **Reutilización:** uso de componentes en distintos proyectos.

### Ejemplo

Un sistema web puede escalar agregando servidores y reutilizar módulos como autenticación en varias aplicaciones.

## 7. ATRIBUTOS DE CALIDAD

- **Rendimiento:** rapidez del sistema.
- **Seguridad:** protección de datos.
- **Disponibilidad:** acceso continuo al sistema.
- **Usabilidad:** facilidad de uso.

### Ejemplo

En una app bancaria, se requiere alta seguridad, disponibilidad 24/7 y rapidez en transacciones.

## 8. TRADE-OFFS EN ARQUITECTURA

Las decisiones implican equilibrar atributos:

- Más seguridad → menor rendimiento
- Más disponibilidad → mayor costo
- Mejor usabilidad → mayor tiempo de desarrollo

El arquitecto debe priorizar según el contexto.

# ESTILOS Y PATRONES ARQUITECTÓNICOS

## 9. ESTILOS PRINCIPALES

- **Arquitectura en capas:** organiza el sistema por niveles.

- **Cliente-servidor:** separación entre usuario y servidor.
- **SOA:** servicios independientes que se comunican.
- **Microservicios:** servicios pequeños y autónomos.

## 10. PATRONES COMUNES

- **MVC:** separa lógica, vista y control.
- **MVVM:** mejora la interacción con la interfaz.
- **Repository:** gestiona el acceso a datos.
- **Pipes and Filters:** procesa datos en etapas.

## 11. SELECCIÓN DE ESTILOS Y PATRONES

Depende de:

- Requisitos del sistema
- Atributos de calidad
- Recursos disponibles
- Contexto del proyecto

# MODELADO Y DOCUMENTACIÓN

## 12. UML Y MODELO C4

- **UML:** lenguaje estándar con diagramas estructurales y dinámicos.
- **C4:** modelo moderno dividido en contexto, contenedores, componentes y código.

Ambos permiten representar la arquitectura de forma clara.

## 13. DIAGRAMAS UML IMPORTANTES

- Casos de uso → funcionalidades
- Componentes → estructura
- Despliegue → infraestructura
- Secuencia → interacción

## **14. MODELO C4**

- Contexto → visión general
- Contenedores → aplicaciones
- Componentes → módulos internos
- Código → detalles técnicos

## **15. DOCUMENTACIÓN SEGÚN ISO 42010**

Incluye:

- Stakeholders
- Requisitos
- Vistas arquitectónicas
- Relación entre componentes
- Justificación de decisiones

## CONCLUSIONES

1. La arquitectura de software es la base que permite diseñar sistemas organizados, escalables y sostenibles en el tiempo.
2. El uso de principios, estándares y patrones facilita el desarrollo de software de calidad, reduciendo errores y mejorando la comunicación entre equipos.
3. Comprender estos fundamentos permite a los estudiantes prepararse para entornos reales, donde las decisiones arquitectónicas impactan directamente en el éxito del sistema.